This Kerberos Never Sleeps

Copyright © 2023 Liran Ma, Texas Christian University.
The development of this document is partially funded by the National Science Foundation's and SaTC program under award No. DGE-1912755. and Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation. A copy of the license can be found at http://www.gnu.org/licenses/fdl.html.

Contents

1 Lab Prep **1**

2 Kerberos Server **2**

3 SSH Server (An Service Provider) **4**

4 SSH Client (A Client Requesting Service) **5**

Overview

Kerberos is a network authentication protocol. It is designed to provide strong authentication for client/server applications by using secret-key cryptography.

Kerberos works on the basis of tickets to allow nodes communicating over a non-secure network to prove their identity to one another in a secure manner. Its designers aimed it primarily at a client–server model, and it provides mutual authentication—both the user and the server verify each other's identity. Kerberos protocol messages are protected against eavesdropping and replay attacks.

1 Lab Prep

In a Kerberos environment, the authentication service logically separates into three parts: A database (db), the Authentication Server (AS), and the Ticket Granting Server (TGS). These three parts, in turn, exist in a single server called the Key Distribution Center (KDC).

Download lab related files from GitHub. Go to your local GitHub folder, and pull the files using the git command as follows.

```
git pull https://github.com/liranmatcu/Docker_Security_Labs.git
# Use ``git clone'' if you have not cloned the project before
```

Examine the docker–composer.yml file inside the "kerberos_ssh" folder. You can see that we will utilize three different containers to represent the three entities in Kerberos (i.e., client, server, and KDC). To be specific, they are ssh-client, ssh-server, and kdc-server containers in our lab.

Remove previous containers and their associate networks to avoid possible interference.

```
docker network prune
docker container prune
```

2 Kerberos Server

Open a terminal on your local machine and issue the following commands to start a Kerberos server container.

```
cd  Docker_Security_Labs/kerberos_ssh
docker-compose up -d kdc-server
```

Check the Dockerfile for more information about users and their passwords if needed. The container's IP address is 10.0.103.2 and hostname is kdc-server, which is specified in the docker-compose.yml file.

Once the container started, switch into the container via

```
docker exec -it kdc-server bash
```

From within the container, start the kadmin shell using the command kadmin.local.

```
kadmin.local
```

You can list the principals with the "listprincs" command as below. An example is shown in Fig. 1. In our example, the domain name is EUREKA.LABS. All the hosts and servers in this lab are within this domain.

```
listprincs
```

Using kadmin, we will now **add principals** for the SSH server and client. In order to add the principal for the SSH server, we first need to identify the host name of the SSH server container. In our case, this will be ssh-server.

```
addprinc -randkey host/ssh-server@EUREKA.LABS
```

```
root@kdc-server:/# kadmin.local
Authenticating as principal root/admin@EUREKA.LABS with password.
kadmin.local:  listprincs
K/M@EUREKA.LABS
kadmin/admin@EUREKA.LABS
kadmin/changepw@EUREKA.LABS
kadmin/kdc-server@EUREKA.LABS
krbtgt/EUREKA.LABS@EUREKA.LABS
noPermissions@EUREKA.LABS
```

Figure 1: kadmin.local with listprincs.

Next, we will create the keytab file for the service principal that will need to be placed on the SSH server container.

```
ktadd -k /sshserver.keytab host/ssh-server@EUREKA.LABS
```

The command above will save the keytab file as sshserver.keytab in the root (/) of the KDC server container.

Now we will add the principal for the (future) user on the SSH server. The username here will need to be identical to the username on the SSH server. Here, we will simply use student as the username.

```
addprinc -randkey student
ktadd -k /student.keytab student@EUREKA.LABS
```

An example is shown in Fig. 2.

```
kadmin.local:  addprinc -randkey host/ssh-server@EUREKA.LABS
WARNING: no policy specified for host/ssh-server@EUREKA.LABS; defaulting to no policy
Principal "host/ssh-server@EUREKA.LABS" created.
kadmin.local:  ktadd -k /sshserver.keytab host/ssh-server@EUREKA.LABS
Entry for principal host/ssh-server@EUREKA.LABS with kvno 2, encryption type aes256-c
kadmin.local:  addprinc -randkey student
WARNING: no policy specified for student@EUREKA.LABS; defaulting to no policy
Principal "student@EUREKA.LABS" created.
kadmin.local:  listprincs
K/M@EUREKA.LABS
host/ssh-server@EUREKA.LABS
kadmin/admin@EUREKA.LABS
kadmin/changepw@EUREKA.LABS
kadmin/kdc-server@EUREKA.LABS
krbtgt/EUREKA.LABS@EUREKA.LABS
noPermissions@EUREKA.LABS
student@EUREKA.LABS
```

Figure 2: After adding principal for the student user.

Once the service and user principals are created, copy the keytab file(s) from the KDC container to your local machine.

```
docker cp  kdc-server:/sshserver.keytab  ./sshserver.keytab

docker cp  kdc-server:/student.keytab    ./student.keytab
```

The “sshserver.keytab” will need to be copied to the SSH server, while the “student.keytab” to the SSH client.

3 SSH Server (An Service Provider)

Start SSH server container as follows.

```
cd  Docker_Security_Labs/kerberos_ssh
docker-compose up -d ssh-server
```

Next, we will copy the sshserver.keytab that we created earlier into the SSH server container under the path /etc/krb5.keytab.

```
docker  cp  ./sshserver.keytab  ssh-server:/etc/krb5.keytab
```

Switch into the ssh-server container via

```
docker exec -it ssh-server bash
```

Add the student user who will utilize Kerberos to log onto the SSH server from a SSH client.

```
adduser student
```

You will be prompted for a password for the student user and other information. Write down the password that you have used.

3.0.1 The student user can log into the SSH server from a client using this password, which is common when Kerberos is not available. If Kerberos is enabled, will the student user still be required to use this password? Think about this question now and you can answer it once you complete the lab.

In order to enable the SSH server to accept Kerberos tickets as part of the authentication, we must enable the following configurations in **/etc/ssh/sshd_config**. Note that we have pre-configured this file in our containers. You may not need to change it unless you encounter errors.

```
GSSAPIAuthentication yes
GSSAPICleanupCredentials yes
```

We will now configure the **/etc/krb5.conf** file used by the client to establish communication with the KDC server. Note that we have pre-configured this file in our containers. You may not need to change it unless you encounter errors (Docker may show different behavior under different operating systems).

```
 [libdefaults]
    default_realm = EUREKA.LABS
    forwardable = TRUE

[realms]
    EUREKA.LABS = {
```

```
            kdc_ports = 88
            kadmind_port = 749
            kdc = kdc-server
            admin_server = kdc-server
    }

[domain_realm]
        host.docker.internal = EUREKA.LABS
```

Configure /etc/hosts by **adding** the mapping for the ssh-client and kdc-server containers' host name and IP.

```
10.0.103.2      kdc-server
10.0.103.4      ssh-client
```

4 SSH Client (A Client Requesting Service)

Start the SSH client from another terminal (this should the third terminal opened for this lab) as follows:

```
cd  Docker_Security_Labs/kerberos_ssh
docker-compose up -d ssh-client
```

Copy the student.keytab to the SSH client container:

```
docker cp  ./student.keytab  ssh-client:/student.keytab
```

Switch into the ssh-server container via

```
docker exec -it ssh-client bash
```

As a check, we will attempt to SSH directly to the server **with the username and password**. For this, we will use the host name of the server container, in our case, simply ssh-server.

```
ssh student@ssh-server
```

You should be able to login via supplying the correct password. After logging in, please log out by issuing the "exit" command and return to ssh-client container.

```
exit
```

We will now configure the /etc/krb5.conf file used by the client to establish communication with the KDC server. Note that we have pre-configured this file in our containers. You may not need to change it unless you encounter errors.

```
 [libdefaults]
    default_realm = EUREKA.LABS
    forwardable = TRUE

[realms]
    EUREKA.LABS = {
            kdc_ports = 88
            kadmind_port = 749
            kdc = kdc-server
            admin_server = kdc-server
    }

[domain_realm]
        host.docker.internal = EUREKA.LABS
```

Next, configure the SSH client to attempt ticket-based authentication. This may be configured in either /etc/ssh/ssh_config globally or in /.ssh/config for a specific user. The key line to add is the one below, which enables the client to use the tickets. Note that we have pre-configured this file in our SSH client container.

```
GSSAPIAuthentication yes
```

There may be additional GSSAPI* options that you may want to configure depending on your preferences. For our example, we only use the minimum requirement.

Configure /etc/hosts by **adding** the mapping for the ssh-server and kdc-server containers' host name and IP.

```
10.0.103.3      ssh-server
10.0.103.2      kdc-server
```

We can now initialize our Kerberos ticket using kinit.

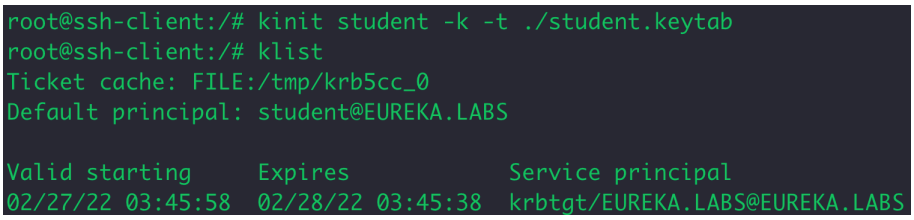
```
kinit student -k -t student.keytab
```

If the above kinit command returns without connection errors, this means that the client was able to reach the KDC successfully based on the krb5.conf settings.

After initializing the TGT ticket for the user principal, you can verify if the ticket was cached correctly using klist.

```
klist
```

An example is shown in Fig. 3.



```
root@ssh-client:/# kinit student -k -t ./student.keytab
root@ssh-client:/# klist
Ticket cache: FILE:/tmp/krb5cc_0
Default principal: student@EUREKA.LABS

Valid starting     Expires            Service principal
02/27/22 03:45:58  02/28/22 03:45:38  krbtgt/EUREKA.LABS@EUREKA.LABS
```

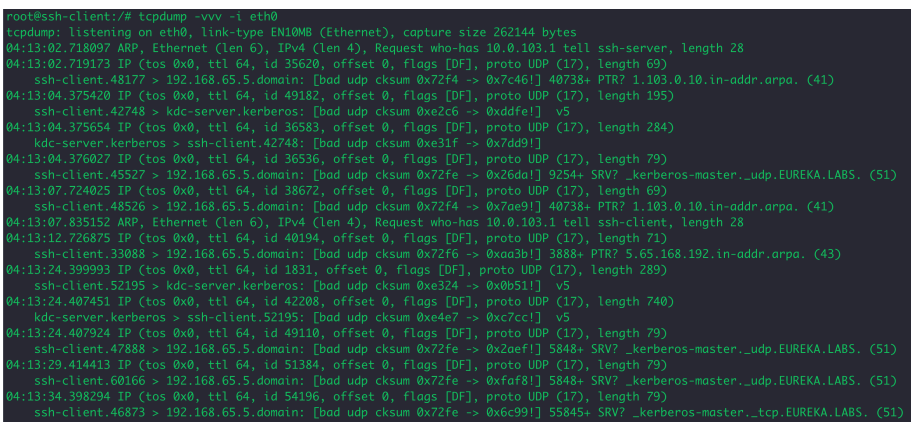
Figure 3: klist command results.

4.0.1 Explain what kind of ticket it is in Kerberos. Is it granted by the Authentication Server (AS) or the Ticket Granting Server (TGS)?

If interested, you can observe Kerberos activities (message exchanges) using to tcpdump on the SSH client container as follows.

```
tcpdump -vvv -i eth0
```

You should be able to see communications between the SSH client and the KDC server, which was the procedure to get the aforementioned ticket. An example is shown in Fig. 4.



```
root@ssh-client:/# tcpdump -vvv -i eth0
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
04:13:02.718097 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.0.103.1 tell ssh-server, length 28
04:13:02.719173 IP (tos 0x0, ttl 64, id 35620, offset 0, flags [DF], proto UDP (17), length 69)
    ssh-client.48177 > 192.168.65.5.domain: [bad udp cksum 0x72f4 -> 0x7c46!] 40738+ PTR? 1.103.0.10.in-addr.arpa. (41)
04:13:04.375420 IP (tos 0x0, ttl 64, id 49182, offset 0, flags [DF], proto UDP (17), length 195)
    ssh-client.42748 > kdc-server.kerberos: [bad udp cksum 0xe2c6 -> 0xddfe!]  v5
04:13:04.375654 IP (tos 0x0, ttl 64, id 36583, offset 0, flags [DF], proto UDP (17), length 284)
    kdc-server.kerberos > ssh-client.42748: [bad udp cksum 0xe31f -> 0x7dd9!]
04:13:04.376027 IP (tos 0x0, ttl 64, id 36536, offset 0, flags [DF], proto UDP (17), length 79)
    ssh-client.45527 > 192.168.65.5.domain: [bad udp cksum 0x72fe -> 0x26da!] 9254+ SRV? _kerberos-master._udp.EUREKA.LABS. (51)
04:13:07.724025 IP (tos 0x0, ttl 64, id 38672, offset 0, flags [DF], proto UDP (17), length 69)
    ssh-client.48526 > 192.168.65.5.domain: [bad udp cksum 0x72f4 -> 0x7ae9!] 40738+ PTR? 1.103.0.10.in-addr.arpa. (41)
04:13:07.835152 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.0.103.1 tell ssh-client, length 28
04:13:12.726875 IP (tos 0x0, ttl 64, id 40194, offset 0, flags [DF], proto UDP (17), length 71)
    ssh-client.33088 > 192.168.65.5.domain: [bad udp cksum 0x72f6 -> 0xaa3b!] 3888+ PTR? 5.65.168.192.in-addr.arpa. (43)
04:13:24.399993 IP (tos 0x0, ttl 64, id 1831, offset 0, flags [DF], proto UDP (17), length 289)
    ssh-client.52195 > kdc-server.kerberos: [bad udp cksum 0xe324 -> 0x0b51!]  v5
04:13:24.407451 IP (tos 0x0, ttl 64, id 42208, offset 0, flags [DF], proto UDP (17), length 740)
    kdc-server.kerberos > ssh-client.52195: [bad udp cksum 0xe4e7 -> 0xc7cc!]  v5
04:13:24.407924 IP (tos 0x0, ttl 64, id 49110, offset 0, flags [DF], proto UDP (17), length 79)
    ssh-client.47888 > 192.168.65.5.domain: [bad udp cksum 0x72fe -> 0x2aef!] 5848+ SRV? _kerberos-master._udp.EUREKA.LABS. (51)
04:13:29.414413 IP (tos 0x0, ttl 64, id 51384, offset 0, flags [DF], proto UDP (17), length 79)
    ssh-client.60166 > 192.168.65.5.domain: [bad udp cksum 0x72fe -> 0xfaf8!] 5848+ SRV? _kerberos-master._udp.EUREKA.LABS. (51)
04:13:34.398294 IP (tos 0x0, ttl 64, id 54196, offset 0, flags [DF], proto UDP (17), length 79)
    ssh-client.46873 > 192.168.65.5.domain: [bad udp cksum 0x72fe -> 0x6c99!] 55845+ SRV? _kerberos-master._tcp.EUREKA.LABS. (51)
```

Figure 4: Tcpdump on client during kinit.

Finally, we can now connect to our SSH server using Kerberos tickets and without needing a password.

```
ssh -vvv student@ssh-server
```

If your Kerberos setup works, you should be able to login without supplying a password. Next, answer the following questions.

4.0.2 Provide a screenshot of your ssh connection results (login as student without a password).

4.0.3 What are the pros and cons of this type of authentication approach?